

INSTITUTE OF AERONAUTICAL ENGINEERING



AN ASSIGNMENT REPORT OF

Software Testing Methodologies

NAME: Thakur Vedika

ROLL NO: 23951A05CG

BRANCH: CSE-E

COURSE CODE: AITD22

FACULTY: Ms. R Prashanthi

FACULTY ID:IARE11190

Scan Properties

Sources Found	0
Words	325
Characters	2296

[View More Details](#)

Plagiarism
0%

0%
Exact Match

0%
Partial Match

Unique
100%

Q1. Explain the purpose of testing.

Answer:

Purpose of Testing

Testing is a systematic process used in software development to evaluate a system or component to determine whether it satisfies specified requirements and works as expected. The primary purpose of testing is to ensure the quality, reliability, and performance of the software before it is delivered to users.

The main purposes of testing are explained below:

1. To Detect Defects

The fundamental purpose of testing is to identify errors, bugs, or defects in the software. Detecting defects early in the development cycle reduces the cost and effort required to fix them later.

2. To Verify Requirements

Testing ensures that the software meets the functional and non-functional requirements specified in documents such as the Software Requirements Specification (SRS). It verifies that the system behaves according to the defined requirements.

3. To Validate the Product

Testing confirms that the developed software fulfills the user's needs and



Congratulations !

No Plagiarism Found

Q1. Explain the purpose of testing.

Answer:

Purpose of Testing

Testing is a systematic process used in software development to evaluate a system or component to determine whether it satisfies specified requirements and works as expected. The primary purpose of testing is to ensure the quality, reliability, and performance of the software before it is delivered to users.

The main purposes of testing are explained below:

1. To Detect Defects

The fundamental purpose of testing is to identify errors, bugs, or defects in the software. Detecting defects early in the development cycle reduces the cost and effort required to fix them later.

2. To Verify Requirements

Testing ensures that the software meets the functional and non-functional requirements specified in documents such as the Software Requirements Specification (SRS). It verifies that the system behaves according to the defined requirements.

3. To Validate the Product

Testing confirms that the developed software fulfills the user's needs and expectations. It checks whether the product solves the real-world problem it was designed for.

4. To Ensure Quality

Quality includes reliability, usability, efficiency, maintainability, and security. Testing evaluates these quality attributes to ensure the product meets industry standards.

5. To Improve Reliability

Through various levels of testing (unit testing, integration testing, system testing, acceptance testing), the system becomes more stable and dependable.

6. To Reduce Risk

Testing helps in identifying potential risks such as security vulnerabilities, performance bottlenecks, or system failures before deployment, thereby minimizing business risks.

7. To Enhance Performance

Performance testing ensures that the software performs efficiently under expected workloads and stress conditions.

8. To Ensure Security

Security testing checks for vulnerabilities, unauthorized access, and data breaches, ensuring the safety of user data.

9. To Provide Confidence to Stakeholders

Successful testing builds confidence among developers, clients, and end-users that the system is reliable and ready for deployment.

10. To Support Maintenance and Future Enhancements

Testing ensures that new changes or updates do not negatively affect existing functionalities (regression testing).

Q2. Importance of testing early.

Answer:

Importance of Testing Early (Early Testing / Shift-Left Testing)

Testing early in the Software Development Life Cycle (SDLC) is one of the most important principles of software engineering. This approach is commonly known as **Shift-Left Testing**, where testing activities start from the requirement and design phases rather than waiting until coding is completed.

The importance of early testing is explained below:

1. Reduces Cost of Defect Fixing

Defects found in the requirements or design phase are much cheaper to fix compared to those found after deployment.

The later a bug is discovered, the more expensive and complex it becomes to correct.

Example:

- Fixing a requirement error during analysis → very low cost

- Fixing the same error after release → very high cost

2. Prevents Defect Propagation

If errors are not detected early, they move to later stages and affect multiple modules. Early testing prevents small issues from becoming major system failures.

3. Improves Software Quality

Early reviews, inspections, and validation ensure that requirements are clear and correct. This results in fewer defects in the final product and improves overall quality.

4. Better Requirement Understanding

Testing activities such as requirement reviews and walkthroughs help identify:

- Missing requirements
- Ambiguous statements
- Conflicting requirements

This ensures clarity before development begins.

5. Saves Time

Fixing defects early reduces rework. Developers do not need to redesign or rewrite large portions of code later, saving development time.

6. Reduces Project Risk

Early testing helps identify technical risks, design flaws, and performance concerns before they become serious problems, minimizing project failure risk.

7. Improves Customer Satisfaction

When fewer defects reach production, users experience a stable and reliable system, increasing customer trust and satisfaction.

8. Supports Agile and DevOps Practices

Modern methodologies like Agile and DevOps emphasize continuous testing and integration. Early testing aligns well with these approaches by ensuring quick feedback cycles.

9. Enhances Team Collaboration

When testing starts early, testers, developers, and analysts collaborate from the beginning. This improves communication and reduces misunderstandings.

Q3. Black-box vs White-box testing.

Answer:

Black-Box Testing vs White-Box Testing

Black-box testing and white-box testing are two fundamental software testing techniques. They differ mainly in whether the tester has knowledge of the internal structure of the code.

Black-Box Testing

Definition

Black-box testing is a testing technique where the tester checks the functionality of the software **without knowing the internal code structure**. The system is treated as a "black box."

Focus

- Inputs and outputs
- Functional requirements
- User behavior

Key Characteristics

- No knowledge of source code required
- Based on requirement specifications
- Tests what the system does

Techniques Used

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing

Example

If testing a login page:

- Enter valid username and password → Should log in
- Enter invalid credentials → Should show error message

The tester does not check how the code processes authentication internally.

Advantages

- Does not require programming knowledge
- Simulates real user perspective
- Good for system and acceptance testing

Disadvantages

- Limited code coverage
- Difficult to detect internal errors

White-Box Testing

Definition

White-box testing is a testing technique where the tester has **complete knowledge of the internal code structure** and tests the internal logic of the program.

Focus

- Code structure
- Branches

- Conditions
- Loops
- Paths

Key Characteristics

- Requires programming knowledge
- Tests how the system works internally

Techniques Used

- Statement Coverage
- Branch Coverage
- Path Coverage
- Condition Coverage

Example

If testing a function that calculates discount:

- Check every if-else condition
- Verify all logical branches
- Ensure all loops execute correctly

Advantages

- High code coverage
- Detects logical errors
- Improves code optimization

Disadvantages

- Requires technical expertise
- Time-consuming for large systems

Q4. Explain error, fault, and failure.

Answer:

Error, Fault, and Failure in Software Testing

In software engineering, the terms error, fault, and failure are closely related but have different meanings. Understanding the difference is important for analyzing defects in a system.

Error

Definition

An error is a human mistake made by a developer, analyst, or designer during software development.

Cause

- Misunderstanding requirements
- Incorrect logic
- Typographical mistakes
- Poor design decisions

Example

- Developer writes the wrong formula for calculating interest.
- Analyst misunderstands the client requirement.

Error occurs at the human level.

Fault (Defect / Bug)

Definition

A fault (also called defect or bug) is the incorrect code or implementation that results from an error.

Cause

- Error in coding
- Design mistake
- Wrong algorithm implementation

Example

If the correct formula is:

$\text{Interest} = \text{Principal} \times \text{Rate} \times \text{Time}$

But the developer writes:

$\text{Interest} = \text{Principal} + \text{Rate} \times \text{Time}$

This incorrect statement in the code is a fault.

Fault exists in the software code.

Failure

Definition

A failure occurs when the software does not perform its intended function during execution.

Cause

- Execution of faulty code
- Unexpected system behavior

Example

If the banking system calculates the wrong interest amount when executed, the incorrect output shown to the user is a failure.

Failure is observed during program execution.

Relationship Between Error, Fault, and Failure

There is a cause-and-effect relationship:

Error (human mistake)

↓

Fault (defect in code)

↓

Failure (incorrect system behavior)

Example Flow:

1. Developer misunderstands requirement → **Error**
2. Writes incorrect condition in code → **Fault**
3. Program crashes when user enters data → **Failure**

Q5. Explain test coverage.

Answer:

Indicates a state transition failure **Test Coverage**

Definition

Test coverage is a measure used in software testing to determine **how much of the software application is tested** by the test cases. It indicates the extent to which the source code, requirements, or functionality are covered during testing. In simple words, test coverage answers the question:

Purpose of Test Coverage

- To ensure maximum code is tested
- To identify untested parts of the application
- To improve software quality
- To reduce risk of undetected defects

Higher test coverage generally means better testing completeness (though 100% coverage does not guarantee zero defects).

Types of Test Coverage

A) Code Coverage

Measures how much of the program source code is executed during testing.

Common types include:

1. **Statement Coverage**
Ensures every statement in the code is executed at least once.
2. **Branch Coverage (Decision Coverage)**
Ensures every branch (true/false) of decision statements is executed.
3. **Condition Coverage**
Ensures each Boolean condition is tested for both true and false values.
4. **Path Coverage**
Ensures all possible execution paths are tested.

B) Requirement Coverage

Measures whether all specified requirements have corresponding test cases.

It ensures:

- No requirement is left untested
- Every feature has at least one test case

Often tracked using a **Requirement Traceability Matrix (RTM)**.

C) Functional Coverage

Measures how many functional features of the system are tested.

D) Risk Coverage

Ensures that high-risk areas of the system are adequately tested.

Q6. Advantages of transaction flow testing.

Answer:

Advantages of Transaction Flow Testing

Definition

Transaction Flow Testing is a black-box testing technique that focuses on verifying the flow of transactions (a sequence of operations or events) through a

system from start to end. It ensures that business processes and workflows function correctly under different conditions.

Advantages of Transaction Flow Testing

1. Ensures Complete Business Process Validation

It verifies the entire transaction path (start → processing → output), ensuring that all steps in a business workflow are executed correctly.

Example:

Online shopping transaction flow:

Login → Select product → Add to cart → Payment → Confirmation.

2. Detects Logical Flow Errors

Helps identify:

- Missing steps
- Incorrect branching
- Wrong decision paths
- Infinite loops

This ensures smooth system behavior.

3. Improves System Reliability

By testing different transaction paths, the system becomes more stable and reliable under real-world usage.

4. Validates User Scenarios

It tests realistic user scenarios instead of isolated functions, making testing more practical and user-oriented.

5. Identifies Dead Ends and Unreachable Paths

Transaction flow diagrams help detect:

- Dead paths (transactions that cannot complete)
- Unreachable states
- Improper exit conditions

Q7. Test case design using dataflow.

Answer:

Test Case Design Using Data Flow Testing

Definition

Data Flow Testing is a white-box testing technique that focuses on the flow of data variables within a program. It examines how variables are **defined, used, and killed (destroyed)** during program execution.

The main goal is to detect errors related to:

- Improper variable initialization
- Incorrect variable usage
- Unused variables
- Redundant definitions

Basic Concepts in Data Flow Testing

Definition (d)

A variable is assigned a value.

Example:

```
int x = 10; // x is defined
```

Use (u)

A variable is used either in:

- **Computation use (c-use)** → Used in calculations
- **Predicate use (p-use)** → Used in conditions

Example:

```
y = x + 5; // c-use of x
```

```
if (x > 0) // p-use of x
```

Kill (k)

When a variable is reassigned or goes out of scope.

Example:

```
x = 20; // previous value of x is killed
```

Data Flow Anomalies

Common errors detected using data flow testing:

Pattern Meaning		Problem
d d	Defined twice without use	Redundant definition
d u	Defined then used	Correct
u d	Used before definition	Error
d k	Defined then killed without use	Unused variable

Steps for Test Case Design Using Data Flow

Step 1: Construct Control Flow Graph (CFG)

Draw nodes representing program statements and edges representing control flow.

Step 2: Identify Definition and Use of Variables

Mark where each variable is defined and used.

Step 3: Determine DU (Definition-Use) Chains

Find paths from variable definition to its use without redefinition.

Step 4: Select Coverage Criteria

Common data flow coverage criteria:

- All-definitions coverage
- All-uses coverage
- All DU-paths coverage

Step 5: Design Test Cases

Create test cases that execute paths covering required DU pairs.

Example

Consider the program:

1. int x;
2. if (a > 5)
3. x = 10;
4. else

5. $x = 20$;

6. `print(x)`;

Analysis:

- Definition of $x \rightarrow$ Line 3 and 5
- Use of $x \rightarrow$ Line 6

DU Pairs:

- $(3 \rightarrow 6)$
- $(5 \rightarrow 6)$

Test Cases:

1. $a = 6 \rightarrow$ Executes line 3 $\rightarrow$ print 10
2. $a = 3 \rightarrow$ Executes line 5 $\rightarrow$ print 20

These test cases ensure all definitions of x are tested.

Q8. Explain transaction paths.

Answer:

Transaction Paths

Definition

A **transaction path** is a complete sequence of steps that a transaction follows from its **starting point (input)** to its **termination point (output)** within a system. It represents the flow of control and data during the execution of a business process.

Transaction paths are mainly used in **Transaction Flow Testing** to verify that all possible business flows work correctly.

What is a Transaction?

A **transaction** is a set of related operations performed as a single unit of work.

Examples:

- Online banking transfer
- Railway ticket booking
- E-commerce order placement
- ATM cash withdrawal

Each of these processes follows a defined flow → called a **transaction path**.

Components of a Transaction Path

A transaction path generally consists of:

1. **Entry Point** – Where the transaction begins
2. **Processing Steps** – Decisions, validations, calculations
3. **Branches** – Alternate flows based on conditions
4. **Exit Point** – Successful completion or termination

Types of Transaction Paths

A) Normal Path (Happy Path)

The ideal flow where everything works correctly.

Example:

Login → Enter valid details → Payment success → Confirmation.

B) Alternate Path

Different valid paths depending on user input.

Example:

Payment by Credit Card / UPI / Net Banking.

C) Exception Path

Path followed when an error or invalid condition occurs.

Example:

Invalid password → Error message → Retry.

Example of Transaction Path

Consider an ATM Withdrawal:

1. Insert card
2. Enter PIN
3. Validate PIN
 - If correct → Proceed
 - If incorrect → Show error
4. Enter withdrawal amount
5. Check balance
 - If sufficient → Dispense cash
 - If insufficient → Show message

Each possible route from start to end is a **transaction path**.

Importance of Transaction Paths in Testing

- Ensures complete business workflow testing

- Detects missing or incorrect logic
- Identifies dead paths or unreachable flows
- Improves system reliability
- Validates real-world user scenarios

Q9. Compare transaction and path testing.

Answer:

Comparison: Transaction Testing vs Path Testing

Transaction Testing and Path Testing are both structural testing techniques, but they differ in focus, level, and purpose.

Transaction Testing

Definition

Transaction Testing focuses on testing the **complete business transaction flows** in a system from start to finish. It verifies whether all possible transaction paths in a business process work correctly.

Focus

- Business workflows
- User operations
- System-level processes

Used In

- System testing
- Integration testing
- Black-box testing

Example

In an online shopping system:

Login → Select product → Add to cart → Payment → Order confirmation.

Each possible workflow (success, failure, alternate payment method) is tested.

Path Testing

Definition

Path Testing is a **white-box testing technique** that ensures all possible execution paths in the program code are tested at least once.

Focus

- Control flow of the program
- Branches and conditions
- Logical decisions

Used In

- Unit testing
- Code-level testing

Example

Testing all possible paths in an if-else or loop structure to ensure every branch executes.

Key Differences

Basis	Transaction Testing	Path Testing
Focus	Business process flow	Program control flow
Testing Type	Mostly Black-box	White-box
Level	System/Integration level	Unit level
Based On	Transaction flow diagrams	Control flow graphs
Objective	Validate user workflows	Cover all code paths
Knowledge of Code	Not required	Required
Example	Banking fund transfer process	Testing all if-else branches

Q10. Explain variable lifecycle.

Answer:

Variable Lifecycle in Software Testing (Data Flow Concept)

Definition

The **variable lifecycle** refers to the different stages a variable goes through during program execution — from its **creation (definition)** to its **termination (destruction or kill)**.

Understanding the variable lifecycle is important in **data flow testing**, as it helps detect errors related to improper initialization, incorrect usage, and redundant assignments.

Stages of Variable Lifecycle

Declaration

The variable is introduced in the program with a data type.

Example:

```
int x;
```

At this stage, memory is allocated, but the variable may not have a meaningful value.

Definition (Initialization)

The variable is assigned a value for the first time.

```
x = 10;
```

Now, x contains a valid value.

☞ In data flow terms, this is called **Definition (d)**.

Use

The variable is used in computations or conditions.

There are two types of uses:

- **Computation Use (c-use)**
Used in calculations
- $y = x + 5;$
- **Predicate Use (p-use)**
Used in decision/condition
- $\text{if } (x > 0)$

Redefinition

The variable is assigned a new value, replacing the old one.

```
x = 20;
```

The previous value of x is overwritten.

Kill (Destruction)

The variable's lifecycle ends when:

- It goes out of scope
- It is explicitly deleted (in dynamic memory languages)
- The program terminates

In data flow terms, this is called **Kill (k)**.

Example:

```
{  
    int x = 5;  
} // x is killed here (out of scope)
```

Variable Lifecycle Representation

The lifecycle can be represented as:

Declaration → **Definition** → **Use** → **Redefinition (optional)** → **Kill**

Data Flow Anomalies Related to Lifecycle

Understanding lifecycle helps detect errors such as:

Pattern	Meaning	Problem
Use before Definition (u-d)	Variable used without initialization	Error
Defined but not Used (d-k)	Value assigned but never used	Wasteful
Double Definition (d-d)	Redefined without use	Logical issue